PROBLÈME · ANNALES SPÉ · 1 H 10

Réseaux, chemins augmentants et plus petite bordure

Langage C (ou pseudo-code de même niveau). Un réseau $H = (W, F, \mu, s, t)$: sommets numérotés de 0 à $n+1$, source $s = 0$, puits $t = n+1$, multiplicité $\mu(e) \in \mathbb{N}^*$ pour chaque arc $e \in F$. Fichier `network.h` :

```
#ifndef NETWORK_H
#define NETWORK_H
const unsigned int NMAX = 102;
struct network_s { unsigned int n; unsigned mu[NMAX][NMAX]; };
typedef struct network_s network;
void nw_init(unsigned int n);
void nw_add(int u, int v);
void nw_remove(int u, int v);
void nw_disconnect(void);
void _arr_init(int *T);
bool _bfs_tree(void);
void _residue(void);
#endif
```

Fichier `network.c` : `network H; int A[NMAX];` ; `H.mu[u][v]` vaut $\mu((u,v))$, ou 0 si (u,v) n'est pas un arc.

1. Rôle des lignes `#ifndef`, `#define` et `#endif`.
2. Comment faire interagir `network.h` et `network.c` ?
3. `void nw_init(unsigned int n)` : précondition $n < n_{\max}$ vérifiée par une assertion ; `H` devient le réseau vide à $n+2$ sommets.
4. `void nw_add(int u, int v)` : incrémente $\mu[u][v]$.
5. `void nw_remove(int u, int v)` : retire une occurrence, avec une assertion si l'arc n'existe pas.
6. `void _arr_init(int *T)` : met les $n+2$ cases de `T` à -1.
7. `bool _bfs_tree(void)` : calcule dans `A` une arborescence de plus courts chemins (parcours en largeur depuis s) : `A[s] = s`, `A[v]` = prédécesseur pour v accessible, -1 sinon ; renvoie `true` si t est accessible.

Un ensemble Π de k chemins de s à t est réalisable si chaque arc e est utilisé au plus $\mu(e)$ fois. Le réseau résiduel $\mathrm{Res}(H, \Pi)$ s'obtient en retirant, pour chaque arc (u,v) de chaque chemin, une occurrence de (u,v) et en ajoutant une occurrence de (v,u).

1. `void _residue(void)` : remplace `H` par $\mathrm{Res}(H, \{\alpha\})$, où α est le chemin de s à t dans l'arborescence `A`.
2. Après k appels successifs (chemins $\alpha_1, \dots, \alpha_k$), montrer qu'il existe un ensemble réalisable Π de k chemins tel que le réseau obtenu soit $\mathrm{Res}(H, \Pi)$.
3. `void nw_disconnect(void)` : tant qu'il existe un chemin de s à t, remplacer H par le résiduel le long d'un plus court chemin ; à la fin, `A` est une arborescence de racine s n'atteignant pas t.
4. Montrer que `nw_disconnect` termine (variant inspiré de 9).

Bordure ∂S : arcs $(u, v) \in F$ avec $u \in S$, $v \notin S$; $\mu(\partial S) = \sum_{e \in \partial S} \mu(e)$. Un entourage est un $S \subseteq W$ avec $s \in S$, $t \notin S$.

1. Pour Π réalisable de k chemins et S entourage : $k \leqslant \mu(\partial S)$.

On fixe Π_0 réalisable de k_0 chemins tel que $\mathrm{Res}(H, \Pi_0)$ n'ait plus de chemin de s à t, et $S_0 =$ sommets accessibles depuis s dans $\mathrm{Res}(H, \Pi_0)$.

1. Pour tout arc (u, v) avec $u \in S_0$, $v \notin S_0$: toutes ses $\mu(e)$ occurrences sont utilisées par Π_0.
2. Aucune occurrence d'un arc (v, u) avec $v \notin S_0$, $u \in S_0$ n'est utilisée par Π_0.
3. Montrer $k_0 = \mu(\partial S_0)$.
4. Montrer $k_0 = \sigma(H)$ (plus petite bordure d'un entourage) et que, après `nw_disconnect`, $\{v : A[v] \geqslant 0\}$ est un entourage de plus petite bordure.